Tool-Result Errors in Language-Model Agents

A Controlled Study of Detection, Reliance, and Recovery

Atikur Rahaman

1 Summary

Language-model agents read tool outputs and act on them, but end-to-end benchmarks score the whole episode and cannot show whether an agent notices a wrong price, date, or policy. This proposal describes a controlled within-prefix study of that moment. We will freeze 800 real τ-bench conversation prefixes immediately after a tool observation, keep each prefix identical, and vary only the observation's content across five arms: verbatim JSON, faithful prose, a non-critical change, a plausible critical error, and a critical error plus a contradiction. Three open-weight 7–8B model families will answer every arm under greedy decoding. We will estimate whether the probability of a decision change rises with error severity and how quickly the agent recovers over four subsequent steps. The study will be pre-registered, gated, and budgeted at $23 base with a $50 hard cap. A null result will be reported as a measurement result.

2 Motivation

2.1 The failure that aggregate scores hide

An agent handling a return request calls `get_order_details` and receives a refund amount of $48.50 instead of $84.50. If the agent repeats $48.50 to the customer, the episode may still be recorded as completed. The failure is not a wrong tool call or a planning error. It is a failure to notice that the observation conflicts with the policy or with the rest of the context. A wrong flight date, a closed return window that the policy says is open, or a cancelled reservation that still has seats produces the same shape of failure. Agents are commonly built as interleaved reason–act loops: they read context, choose an action, receive an observation, and repeat [8]. This pattern is what makes tool use useful, and it is also what makes the observation a decision point. The agent's next action is conditional on what the tool returned. If that return is wrong, the agent can propagate the error, re-query the tool, flag the conflict, or choose a different action. Aggregate task scores do not separate these outcomes.

2.2 What existing benchmarks measure

End-to-end agent benchmarks score the final state of an episode. τ-bench compares the database state at the end of a conversation with an annotated goal state and reports task success and pass^k reliability [1]. AgentBench evaluates agents across eight interactive environments [11]. WebArena measures task completion in a realistic web environment [4]. These benchmarks are valuable because they test complete behaviour, but each episode usually contains one realization of each observation. Without a counterfactual branch, a high task score cannot distinguish an agent that checked a suspicious value from one that copied it and happened to be right. Tool-use benchmarks focus on different slices of the problem. ToolLLM trains and evaluates models on a large collection of real-world APIs [2]. MINT evaluates multi-turn tool use with natural-language feedback [3]. The Berkeley Function Calling Leaderboard scores function selection and argument construction, including multi-turn cases [12]. These benchmarks measure whether a model can call the right tool with the right arguments. They do not hold a real observation fixed and vary only its content.

Prior work also shows that context and content influence model behaviour. Performance varies with the position of relevant information in long contexts [9]. Models can agree with a user's

stated belief over a truthful answer, a behaviour known as sycophancy [10]. Tool-returned content can hijack an agent in prompt-injection attacks: AgentDojo evaluates attacks and defences in tool-using environments [6], and InjecAgent benchmarks indirect prompt injections in tool-integrated agents [7]. ToolEmu uses an emulated sandbox to identify risky tool-use behaviour [5]. These studies establish that observation content can change what an agent does. They study position, social pressure, or adversarial instructions. They do not estimate a graded, within-prefix effect of plausible factual errors, and they do not measure how long the effect persists. The gap is therefore specific. We lack a controlled estimate of how the content of a realistic tool observation changes the next decision, whether that change is ordered by the severity of the content error, and how quickly the agent returns to the verbatim-observation decision. This proposal targets that gap with a paired design, a frozen real-prefix dataset, and a pre-registered analysis.

2.3 The pilot: a screening signal

A 60-prefix pilot tested whether the manipulation is runnable. It used synthetic WebArena-lite / τ-style prefixes, `qwen3.5:4b`, greedy decoding, and three collapsed action codes (trust, re-call, flag). The pilot compared a truthful arm (T), a placebo arm (P), and an error arm (E). Table 1 reports the recorded results.

Table 1: Pilot results. The pilot used $N = 60$ synthetic WebArena-lite / τ-style prefixes, `qwen3.5:4b`, greedy decoding, and three collapsed action codes. It is a screening signal, not an estimate of the confirmatory slope.

Quantity	Result
Consult rate	54/60 = 90.0%
Placebo jitter (placebo vs truthful)	0/60 = 0%
Error switch (error vs truthful)	5/60 = 8.3%
Gap, 95% CI	8.3 pp [1.3, 15.3]
McNemar	$b = 5$, $c = 0$; exact $p = .0625$
Human spot-check	12/12 labels agree

These numbers show that the manipulation is runnable, that the model consults observations, and that error content can move the next action. They do not estimate a dose–response slope. The sample is small, the model is a single 4B model, the prefixes are synthetic, the action coding is coarse, and there are no recovery data. The recorded 95% interval is an unpaired Wald interval; the paired McNemar test is the design-matched comparison. That interval is compatible with a 1.3 percentage-point effect, and the McNemar result is marginal at $p = .0625$. A 12-item human spot-check agreed with all 12 labels, but the letter coding missed some silent fact propagation, so 8.3 percentage points is an action-level lower bound rather than a content-sensitivity estimate. The pilot is therefore a screening signal. It is not pooled with the confirmatory sample and is not used to rescue a null confirmatory result.

3 Objective

3.1 Research questions

The primary question is: *conditional on a real tool observation inside a frozen prefix, does changing the observation's content change the agent's next decision, and is the change ordered by the severity of the content error?* Two secondary questions follow. First, when the decision changes, how often is the false value propagated silently rather than flagged or detected? Second, over four subsequent steps, how quickly does the agent recover under transient versus persistent injection?

3.2 Estimand and hypotheses

For prefix p, model m, and severity $s \in \{0, 1, 2, 3\}$, define the switch indicator

$$Y_{p,m,s} = \mathbf{1}[\mathrm{sig}(p, m, s) \neq \mathrm{sig}_V(p)]\,, \tag{1}$$

where $\mathrm{sig}(p, m, s)$ is the decision signature under arm s and $\mathrm{sig}_V(p)$ is the verbatim-observation continuation. The primary model is a conditional (fixed-effects) logistic regression stratified by prefix:

$$\operatorname{logit} \Pr(Y_{p,m,s} = 1) = \alpha_{p,m} + \beta_m s. \tag{2}$$

Here $\alpha_{p,m}$ is a prefix-by-model intercept and β_m is the log-odds change per one-level increase in severity. The pooled model stratifies by prefix $\times$ model and clusters standard errors by prefix. V is excluded from the slope because it differs in format as well as content; $s = 0$ is the placebo that absorbs format jitter.

The pre-registered hypotheses are:

- **H1 (primary).** The pooled slope of a decision switch on severity is positive: $\beta > 0$.
- **H2.** The per-model slope is positive in at least two of three models, with the same sign.
- **H3.** Silent propagation (Y^U) is a non-zero share of the decision changes at $s = 2$ and $s = 3$.
- **H4.** At least 50% of the $s = 3$ risk set recovers by $H = 4$, and the Weibull shape is not rejected at $p < .01$.
- **H5.** The $s = 3$ switch rate exceeds the $s = 1$ rate.

H1 is the primary test. H2 is a replication gate, not a model ranking. H4 is an estimation target: if it fails, the paper reports the recovery curve and restricted mean survival time (RMST). Severity is ordinal, not interval. The linear slope is a summary of the graded contrast; the $s = 2$ versus $s = 3$ contrast is always reported separately because $s = 3$ is intentionally compound. A quadratic term and a categorical-severity likelihood-ratio test are pre-registered diagnostics. If linearity is rejected, the paper reports tier-specific odds ratios and treats the slope as a summary rather than a law.

3.3 Claim boundary

If every gate passes, the study will claim a bounded content-sensitivity and recovery result for three open-weight 7–8B model families on τ-bench airline and retail, estimated within prefix. It will not claim a universal constant or law, a ranking of models by sensitivity or recovery speed, transfer to frontier or API models, invariance across tasks or domains, an effect on end-to-end task success, or external validity to live web and API tools. τ-bench observations are deterministic benchmark outputs, not live services. A null or flat slope is a legitimate outcome. It will be reported as a measurement result at a known precision, with the dataset and the negative result released together. The pilot will not be used to reframe a null confirmatory result as a positive law.

4 Proposed Method and Dataset

4.1 Design: one prefix, five observation arms

The unit is a frozen message prefix ending immediately after a real τ-bench tool observation. The prefix contains the system policy, the user turns, the donor assistant turns, and all prior tool calls and results. The target model produces the next decision; it never generates or repairs the prefix. Every arm is generated from the same prefix, so task difficulty, domain, donor style, and conversation history are held constant. The comparison is within prefix, in the sense of a within-unit causal contrast [14]. The donors are the public gpt-4o and claude-3.5-sonnet τ-bench trajectories. The target models are open-weight 7–8B models. Separating donors from targets avoids a circular design in which the model that produced the history also supplies the outcome.

4.2 ToolResult-800: real prefixes, frozen before outcomes

ToolResult-800 will be built from the public τ-bench historical trajectories (MIT): gpt-4o airline, gpt-4o retail, sonnet-35-new airline, and sonnet-35-new retail. The supply is measured, not assumed. The planning analysis found 1,980 trajectories, 13,605 post-tool candidates, 11,318 JSON-parseable observations, and 6,554 eligible value-used observations across 165 task–domain pairs (airline 1,593; retail 4,961). A cap of 10 prefixes per task–domain yields 1,646, so 800 is feasible; the airline arm is the binding constraint.

Table 2: Planned ToolResult-800 supply funnel. Counts are measured from the public τ-bench trajectories and are planning inputs, not confirmatory results. Profile-tool exclusions are applied after JSON parsing; the value-used filter is applied last.

Stage	Count
Source trajectories	1,980
Post-tool candidates	13,605
JSON-parseable observations	11,318
Profile-tool candidates excluded	3,692
Eligible value-used observations	6,554
airline / retail	1,593 / 4,961
Task–domain pairs	165
Capped at 10 per task–domain	1,646
Selected (primary + reserve)	880
Primary (airline / retail)	800 (400 / 400)
Reserve (airline / retail)	80 (40 / 40)

A prefix is eligible if it contains a real tool message, a reconstructable prefix, at least one scalar value used in the donor's factual continuation, PII-scrubbed content, a replay match of at least 98%, and a lossless $s = 0$ rendering. The main exclusions are profile-tool observations (`get_user_details`), observations with no used field, unreproducible observations, decision-critical PII, and information loss under the prose renderer. Replay is the step that makes an observation a real environment output. The harness loads the pinned task database, executes the prefix's tool calls from the initial state, and compares the final observation with the historical one using a hash and normalized values. The 98% threshold is not relaxed. Replay also makes the four-step recovery continuation possible, because the episode can continue against the real environment rather than the donor transcript.

Sampling is stratified by domain, donor, step-index bucket, and tool family, with seed 20261108. The study selects 880 prefixes: 800 primary (400 airline, 400 retail) plus 80 reserve (40 airline, 40 retail). The sample is frozen before any target-model output is generated. The manifest records prefix IDs, message hashes, observation hashes, arm assignments, perturbation-record hashes, the renderer version, the field-registry hash, and the preregistration hash. Every analysis table must trace to a manifest row and a ledger row.

Raw τ-bench trajectories may contain names, emails, addresses, phone numbers, and payment identifiers. Profile tools are excluded, direct identifiers become stable synthetic tokens, and a prefix is excluded if decision-relevant content is itself PII. A 40-item human audit checks eligibility, PII, relevance, and replay before freeze; a separate re-scan runs before release. Raw trajectories, weights, caches, credentials, and run logs never enter the repository.

4.3 Manipulation and rendering

All arms use a real observation from the same prefix. V is the exact observation as raw JSON. The four prose arms use one deterministic key–value/bullet template; only content changes. No LLM-generated text enters the manipulation. A script flattens the V value set and the $s = 0$ value set and verifies equality; any loss excludes the prefix. Every perturbation record stores the prefix

ID, JSON path, true value, false value, criticality label, expected action change, and renderer version. PII fields are never perturbed.

Table 3: Observation arms. All prose arms use the same deterministic template. V is excluded from the primary slope because it differs in format as well as content.

Arm	Level	What changes	Role
V	anchor	Exact real observation, raw JSON.	Format/jitter anchor
$s = 0$	0	Faithful prose; all values preserved.	Placebo
$s = 1$	1	One non-critical field changed or omitted.	Negative control
$s = 2$	2	One critical field changed to a plausible wrong value.	Dose
$s = 3$	3	Critical wrong value plus a second detectable contradiction.	Dose (compound)

The construction rules are pre-specified. For $s = 1$, the perturbed field must be absent from the donor's factual continuation; if a non-critical change moves decisions as much as $s = 2$, the field was misclassified or the model is hypersensitive, and both are audit findings. For $s = 2$, the wrong value must be plausible for the entity and must change the correct action if trusted; absurd values are rejected. For $s = 3$, the critical error is combined with a second inconsistency that a careful reader can detect, such as "cancelled" while seats are available or "non-refundable" inside the refund window. Calibration checks that $s = 3$ is not a ceiling. A worked example makes the coding concrete. This is an illustrative construction, not a harvested record. A user asks to return headphones from order W1842. The tool returns status "delivered", delivery three days ago, a 30-day return window that is open, a refund of $84.50, and a product rating of 4.4. Under $s = 2$, the refund becomes $48.50. If the agent calls `initiate_return` with the same arguments but tells the customer $48.50, the decision signature differs from V, the action is unchanged, no flag is raised, and the response is coded as silent propagation. Under $s = 3$, the refund is $48.50 and `return_window_open` is false although the item arrived three days ago. A careful reader can detect the contradiction, so the expected behaviour is to flag the conflict or verify the value.

4.4 Models and inference

The confirmatory run uses three open-weight model families: Qwen2.5-7B-Instruct-AWQ, Mistral-7B-Instruct-v0.3-AWQ, and Llama-3.1-8B-Instruct-AWQ. All are 4-bit AWQ quantized and served one at a time under greedy decoding on a single 24 GB RTX 3090 or 4090 rental, with batch sizes of 8–16 and a pinned 8k context. The local RTX 3060 (6 GB) is used only for imports, smoke tests, renderer checks, and tiny forward passes. Model revisions are pinned before calibration. If Llama-3.1 access fails, the pre-registered substitutes are InternLM2.5-7B-Chat or Yi-1.5-9B-Chat, recorded before calibration and never selected after seeing target-model outcomes. Calibration is a harness check, not a result. It uses 60 prefixes (30 per domain), five arms, and three models, for 900 generations. The per-model gates are: parse rate at least 95%, placebo jitter at most 20%, $s = 3$ switch rate between 10% and 80%, and format refusal below 5%. No main-run spend occurs before calibration passes. Calibration prefixes are disjoint from the main 800 and stay out of the primary analysis.

4.5 Outcomes and analysis

A decision signature is the tool name plus canonicalized JSON arguments, or the final answer's action verb plus decision-critical slots. Canonicalization sorts keys, normalizes numbers, and lower-cases entities. The verbatim V continuation is the reference. The outcome labels are:

- Y: any sensitivity; the signature differs from the verbatim V continuation.
- Y^U: silent propagation; the false value of a critical field appears in decision content or arguments, with no flag. Not scored for $s = 1$.
- Y^F: detection; an explicit flag, doubt, "inconsistent", "verify", or "cannot trust".

- Y^A: action switch; the tool name or canonicalized arguments differ from V.
- Decomposition: $Y = Y^A \vee Y^F \vee Y^U$; strict silent propagation is $Y^U \wedge \neg Y^A \wedge \neg Y^F$.

Recovery uses the risk set of prefixes whose decision moved at the injected step ($Y^U = 1$ or $Y^A = 1$) and did not flag there, separately for $s = 2$ and $s = 3$. The episode continues for $H = 4$ steps against the real environment. Recovery at step h is the first step where the signature matches V (R1: correct decision) or the model explicitly flags (R2: detection). A sequence that does neither by $H = 4$ is censored. Transient injection is primary; persistent injection corrupts the same field on 25% of prefixes and requires the model to flag or work around the corruption rather than merely re-query for truth.

The primary analysis is the conditional logistic model in Equation 2. The pooled model stratifies by prefix × model and clusters by prefix. The fixed testing sequence is: pooled β; per-model β with Holm correction across three models; descriptive silent propagation; recovery; exploratory heterogeneity. No model ranking is a confirmatory claim. Linearity is checked with a quadratic term and a categorical-severity likelihood-ratio test. If linearity is rejected, tier-specific odds ratios are reported and the slope is described as a summary. The design assumes a 4% placebo switch rate and a random prefix intercept with standard deviation 1.0. At $N = 800$, the per-model minimum detectable effect is 0.203 logit per level (odds ratio 1.22) at 80% power and $\alpha = .05$; the pooled minimum detectable effect is at most 0.15. Power is not the binding constraint. Construct validity is.

Recovery is analysed with a discrete-time exponential survival model with interval censoring:

$$\Pr(\text{recover at } h \mid \text{not yet}) = 1 - e^{-\mu}, \qquad t_{1/2} = \frac{\ln 2}{\mu}, \tag{3}$$

over $H = 4$. Kaplan–Meier and restricted mean survival time (RMST) are reported regardless. A Weibull shape likelihood-ratio test at $p < .01$ checks the exponential assumption. If the shape rejects or fewer than 50% of the risk set recovers by $H = 4$, $t_{1/2}$ is labelled exploratory and the curve and RMST are the result.

Human labels validate automatic coding; they do not replace it. The severity audit uses 120 observations (30 per level) with three blind raters and requires Krippendorff's $\alpha \geq .75$ [13], a Jonckheere–Terpstra trend test at $p < .01$, at least 90% of $s = 1$ items rated non-decision-changing, at least 90% of $s = 2$ and $s = 3$ items rated decision-changing, and no more than 20% of $s = 3$ items rated absurd. The outcome audit uses 160 responses with two blind raters and requires Cohen's $\kappa \geq .80$, silent-propagation positive predictive value at least .90, parser failure below 1%, and seed-repeat stability at least .90. The recovery audit uses 80 sequences and requires $\kappa \geq .75$. If outcome κ falls below .70 after one rubric revision, the study downgrades to action-level Y^A only and does not make the content-sensitivity claim. Gates are sequential and frozen before the main run. A gate cannot be skipped, relaxed after seeing data, or re-run on a different sample without a logged amendment. Table 4 lists the criteria and the consequence of failure.

Table 4: Sequential gates and stop rules. A failed gate changes the claim, not the data.

Gate	When	Pass criterion	If it fails
G0	End of week 2	At least 880 eligible; at least 440 per domain before audit and 400 after audit; replay at least 98%; 40-item audit at least 36/40.	Fall back to 500 prefixes and 2 models, or stop.
G1	Week 3	Parse at least 95%; placebo jitter at most 20%; $s = 3$ switch 10–80%; refusal below 5% per model.	Fix prompt or parser and rerun; no main spend.
G2	Weeks 4–7	Stop at $22 spend or 45 GPU-hours, whichever comes first; ledger complete.	Analyse completed shards; disclose the shortfall.
G3	Weeks 8–10	Pooled $\beta > 0$; positive in at least 2 of 3 models with the same sign; severity $\alpha \geq .75$; outcome $\kappa \geq .80$ (or adjudicated at least .70 with downgrade); recovery $\kappa \geq .75$.	Null or measurement paper; no content-sensitivity claim.
G4	Weeks 9–10	At least 50% of the $s = 3$ risk set recovers by $H = 4$; Weibull shape not rejected at $p \geq .01$.	Report RMST and the curve; $t_{1/2}$ exploratory only.

4.6 Implementation plan

The study runs in six overlapping phases over 18 weeks with a part-time team of three to four people. P0 (weeks 1–2) is the CPU-only harvest and freeze: pin τ-bench, parse, scrub, label relevance, replay, sample 880, render, audit, and freeze the manifest and preregistration. P1 (week 3) is calibration. P2 (weeks 4–7) is the main run. P3 (weeks 6–10) covers labels and analysis, beginning as soon as the first main-run shard lands. P4 (weeks 9–14) is the paper, beginning when the primary slope is estimable. P5 (weeks 15–18) is the ToolResult-800 release. The first step is the ten-day G0 sprint, which is CPU-only, costs $0, and can invalidate the study before any GPU spend.

The main run is 34,800 generations: 12,000 base decisions (800 prefixes $\times$ 5 arms $\times$ 3 models), 19,200 transient recovery steps (800 $\times$ 2 severities $\times$ 4 steps $\times$ 3 models), 2,400 persistent recovery steps (200 $\times$ 1 severity $\times$ 4 steps $\times$ 3 models), and 1,200 jitter repeats (80 $\times$ 5 arms $\times$ 3 models). The runner checkpoints every 40 prefixes, appends one ledger row per generation, and resumes after an interruption without overwriting completed rows. A parser-error rate above 5% in a shard fails fast. The budget is $23 base with a $50 hard cap. The GPU line is budgeted at $15–22 for 35–45 hours on one 24 GB RTX 3090/4090, with a $22 stop; storage and egress are $2–3; contingency is $3–5; annotation is about 200 team judgments at $0; local CPU work is 20–30 hours at $0; and there is no paid API. The repository is currently at scaffold version 0.1.0. No confirmatory data have been collected, the pipeline entry points are stubs, and no gate has been run.

5 Expected Outcome

5.1 If the gates pass

If the gates pass, the study will report a positive pooled slope, per-model slopes, the share of decision changes that are silent propagation, and a recovery curve with a pooled half-life where identifiable. The claim will be bounded to three open-weight 7–8B families, τ-bench airline and retail, one deterministic renderer, four severity levels, and four recovery steps. The result will be conditional on the frozen sample, renderer, and rubric, and every deviation will be reported through the amendments log. A positive result with a substantial silent-propagation share would show that aggregate task scores can hide a real failure mode: an agent can complete an episode

while repeating a wrong value. A graded slope would show that the probability of a decision change depends on how severe the content error is, not merely on whether the format changed. A recovery curve would show whether the effect is transient or persistent, which matters for whether a single verification step is enough.

5.2 If the result is null

A flat or negative slope, a placebo jitter above 20%, an $s = 3$ switch rate outside 10–80%, or recovery in fewer than 50% of the risk set will be reported as a null or measurement result. The paper will state the precision of the estimate and will not reframe exploratory analyses as confirmatory. A well-powered null is useful: it bounds how much observation content matters at a known precision and prevents practitioners from over-investing in content-level checks. The dataset and the null result will still be released, so other groups can replicate or extend the design.

5.3 Deliverables and limitations

The study will produce ToolResult-800 (800 frozen real prefixes plus 80 reserve, with perturbations, arm assignments, renderer version, decision signatures, and recovery traces), frozen manifests with all hashes, one ledger row per generation, a reproducibility package, the preregistration and amendments log, and a paper draft. The contributions are a dataset, an empirical within-prefix estimate, a reusable protocol, and a null-result contribution if the effect is absent.

The main limitations are known in advance. τ-bench tasks are public and may be memorized by target models, so baseline reliance could be inflated; the within-prefix contrast remains valid, but the absolute switch rate may not transfer. The donor prefixes come from gpt-4o and claude-3.5-sonnet, so the target models continue off-policy histories; the design fixes prefixes across arms and stratifies by donor, but it does not make the history on-policy. Severity is a constructed ordinal variable; if $s = 1$ moves decisions as much as $s = 2$, the dose interpretation fails and the study reports that finding. The study covers three 7–8B model families, two domains, one renderer, deterministic tools, and a four-step horizon. It does not claim end-to-end task success, frontier-model behaviour, live-API validity, or a universal law.

6 Application

6.1 Agent reliability

The result will inform concrete engineering choices at the observation boundary. If wrong content moves decisions and silent propagation is common, builders should add verification steps for critical fields, require a second tool call before acting on a suspicious value, surface a confidence flag, or route high-stakes actions to human approval. If the slope is flat, those checks may be less urgent than tool selection, planning, or policy adherence. The study does not recommend a specific product change; it estimates the size of the problem and the conditions under which it appears.

6.2 Evaluation practice

End-to-end success is a useful but coarse metric. Two agents with the same task score can differ sharply in whether they notice a wrong observation. An observation-level sensitivity metric complements task success and can be reported alongside it. ToolResult-800 provides a reproducible test bed for that metric: the same frozen prefixes, arms, renderer, and coding rules can be applied to new models without re-harvesting trajectories. This makes comparisons across model families more interpretable than a single aggregate score.

6.3 Safety and reuse

Wrong amounts, dates, and policies can cause financial loss or unsafe actions. Silent propagation is especially relevant because the transcript looks fluent and the episode may be marked complete. Making this failure mode visible supports incident analysis, audit, and the design of guardrails. A negative result is also useful for accountability: it shows that a proposed guardrail is not addressing

a large effect in this setting, which can free resources for other risks. The protocol can be extended to frontier models, live APIs, other domains, or training interventions in follow-up work, but those questions are outside this study's scope. The release includes a data card, checksums, the preregistration, and the amendments log, so other groups can replicate the analysis or test a different model family. The repository separates code (MIT), documentation and derived data (CC BY 4.0), and upstream τ-bench (MIT); raw trajectories, weights, and credentials are never released.

References

[1] Yao, S., Shinn, N., Razavi, P., and Narasimhan, K. (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv:2406.12045*.

[2] Qin, Y., et al. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv:2307.16789*.

[3] Wang, X., Wang, Z., Liu, J., Chen, Y., Yuan, L., Peng, H., and Ji, H. (2023). MINT: Evaluating LLMs in Multi-turn Interaction with Tools and Language Feedback. *arXiv:2309.10691*.

[4] Zhou, S., et al. (2023). WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv:2307.13854*.

[5] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., and Hashimoto, T. (2023). Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *arXiv:2309.15817*.

[6] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *arXiv:2406.13352*.

[7] Zhan, Q., Liang, Z., Ying, Z., and Kang, D. (2024). InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. *arXiv:2403.02691*.

[8] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv:2210.03629*.

[9] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *arXiv:2307.03172*.

[10] Sharma, M., et al. (2023). Towards Understanding Sycophancy in Language Models. *arXiv:2310.13548*.

[11] Liu, X., et al. (2023). AgentBench: Evaluating LLMs as Agents. *arXiv:2308.03688*.

[12] UC Berkeley Gorilla Team. (2024). Berkeley Function Calling Leaderboard (BFCL). `https://gorilla.cs.berkeley.edu/leaderboard.html`. Accessed 11 November 2026.

[13] Krippendorff, K. (2004). *Content Analysis: An Introduction to Its Methodology* (2nd ed.). Sage Publications.

[14] Hernán, M. A., and Robins, J. M. (2020). *Causal Inference: What If*. Chapman & Hall/CRC.